SQIsign v2.0.1 Specification Review

Feedback from an Independent Rust Implementation

Deirdre Connolly
Selkie Cryptography
durumcrustulum@gmail.com

May 12, 2026 (living document)

Abstract

Each item below describes something that the SQIsign specification [1] (version 2.0.1, dated 2025-07-07) leaves unstated, ambiguous, or implicit, and that we could only resolve by inspecting the C reference implementation. Implementing SQIsign from the v2.0.1 spec alone does not currently produce an implementation that interoperates with the C reference: the gaps below are not edge cases or optimizations, but load-bearing conventions whose absence causes silent KAT failures during signing or verification. We file these as constructive suggestions so that future implementors can achieve interoperability from the spec alone.

Contents

1 Severity scale

Critical	The spec as written produces a non-interoperable implementation: verification rejects valid signatures, or signing produces unverifiable ones. The spec must be amended.
High	An implementor will almost certainly write a bug that passes unit tests but fails KAT or cross-implementation testing. Debugging requires reading the C reference source.
Medium	An implementor may write a bug depending on reasonable but wrong assumptions about conventions. A short note eliminates the ambiguity.
Low	The spec is correct but incomplete. A remark or forward pointer would help.

2 $\mathbb{F}_{p^2}$ square root canonicalization

Affects Algorithm 8.12 (DifferencePoint), and transitively every algorithm that consumes a torsion basis. Critical

Algorithm 8.12 says "compute $\delta = \sqrt{B_{XZ}^2 - B_{XX}B_{ZZ}}$" without specifying which of the two square roots in $\mathbb{F}_{p^2}$ to use. The two roots δ and $-\delta$ produce two *affinely distinct* points x_{P-Q}. These are not projectively equivalent; they are genuinely different points. The wrong choice cascades through Ladder3pt (wrong kernel generator), the challenge isogeny (different curve E_{chl}), and every subsequent verification step. In our implementation the j-invariant of E_{chl} was completely wrong until we matched the C reference's canonicalization.

The spec should define a canonical $\mathbb{F}_{p^2}$ square root. The C reference uses the convention from Aardal et al. [2]: negate the output so that the real part (as an integer in $[0, p)$) is even; if the real part is zero, negate so the imaginary part is even. This should be stated in §8.1 (field arithmetic) as a normative requirement. A note of the form

> *Throughout this document, "compute $\sqrt{a}$" for $a \in \mathbb{F}_{p^2}$ means the unique root r such that $r^2 = a$ and $\mathrm{Re}(r) \bmod 2 = 0$ (or, if $\mathrm{Re}(r) = 0$, $\mathrm{Im}(r) \bmod 2 = 0$).*

would suffice.

3 ProductToTheta coordinate convention

Affects Algorithm 8.37 (ProductToTheta). High

The product theta coordinates are described abstractly via level-2 theta functions. The mapping from Montgomery projective coordinates $(X : Z)$ to product theta coordinates is not stated explicitly. The natural "theta-like" representation for Montgomery arithmetic is $(X - Z,\ X + Z)$, which appears throughout the isogeny literature. The C reference uses raw (X, Z) directly. Using $(X - Z, X + Z)$ produces a completely wrong change-of-basis matrix N.

Algorithm 8.37 should state explicitly that the product theta coordinates are formed as

$$(a, b, c, d) = (X_1X_2,\ X_1Z_2,\ Z_1X_2,\ Z_1Z_2)$$

from Montgomery projective pairs $(X_1 : Z_1)$, $(X_2 : Z_2)$.

4 "Squared theta" definition

Affects Algorithm 8.29, Algorithm 8.34, Algorithm 8.38. High

Several algorithms refer to "squared theta coordinates" without a centralized definition. In the C reference, `to_squared_theta(P)` computes component-wise squaring *followed by a Hadamard transform*: $\tilde{P} = \mathcal{H}(P^2)$. The name suggests only squaring. We omitted the inner Hadamard in the generic isogeny evaluation, computing $\mathcal{H}(\mathrm{inv} \cdot P^2)$ instead of the correct $\mathcal{H}(\mathrm{inv} \cdot \mathcal{H}(P^2))$.

Add a definition early in §8.5:

> *The **squared theta** of a point $P = (a : b : c : d)$ is $\tilde{P} = \mathcal{H}(a^2, b^2, c^2, d^2)$, i.e. component-wise squaring followed by the Hadamard transform.*

5 GluingEval cross-curve products are not explicit

Affects Algorithm 8.39 (GluingEval). High

The product-curve gluing evaluation runs AddComponents on each curve component separately, producing (u_1, v_1, w_1) from curve 1 and (u_2, v_2, w_2) from curve 2, then assembles the output theta point from products of these six values. In the spec's pseudocode the U, V, W vectors are written as products without explicit subscripts on every factor, leaving the reader to infer which factors come from which curve.

The defining property of a product-curve formula is that every multiplication has *one* factor from curve 1 and *one* factor from curve 2. An implementor reading "$U_2 \leftarrow u_1 \cdot w_1$" (same subscript on both factors) cannot tell at the line whether this is a deliberate within-curve product (rare) or a transcription error for the cross-curve product $u_1 \cdot w_2$ (almost always correct). We made exactly this transcription error: the resulting null point passed the on-curve check, ran through the chain without raising, and the codomain failed only at SplittingIsomorphism far downstream.

Recommendation. In Algorithm 8.39, write every product factor with an explicit subscript and verify the cross-curve property holds. A short note of the form "*every multiplication in AddComponents's output assembly takes one factor from each component curve*" would catch the typo class at code review time.

6 Projective inverse of the dual theta null point is not field inversion

Affects Algorithm 8.37 (GluingCodomain) and similar product-theta constructions where "inverse" appears as a sub-step. Medium

The gluing codomain construction (Algorithm 8.37, transitively referenced from Algorithm 8.39) requires the "inverse" of the dual theta null point $(\alpha, \beta, \gamma, 0)$. In projective theta coordinates this is computed via cross-products — pure multiplications producing a vector projectively equivalent to the multiplicative inverse. No field inversion is involved.

The spec uses the word "inverse" without qualifying it, which an implementor can reasonably read as the field-inverse operation. Field-inverting components in $\mathbb{F}_{p^2}$ is both numerically wrong (the projective and field inverses agree only up to scaling that's lost between projective classes) and expensive (each inverse is hundreds of multiplications).

Recommendation. Replace "inverse" in Algorithm 8.37 with "projective inverse via cross-products" and state the closed-form expressions explicitly. In general, in any projective- or theta-coordinate algorithm, "inverse" should be qualified: an actual field inversion at this layer is almost always a transcription error from an affine reference, and is worth flagging in the spec.

7 `hadamard_bool` in the $(2,2)$-chain

Affects Algorithm 8.47 (Isogeny22ChainWithTorsion). High

The algorithm describes each generic step uniformly. The C reference uses three formula variants controlled by boolean flags:

- Normal ($i < n-2$): codomain Hadamard-transformed; eval = $\mathcal{H}(\text{precomp} \cdot \mathcal{H}(P^2))$.
- Penultimate ($i = n-2$): codomain in dual form; eval = $\text{precomp} \cdot \mathcal{H}(P^2)$.
- Ultimate ($i = n-1$): extra input Hadamard, codomain dual; eval = $\text{precomp} \cdot \mathcal{H}(\mathcal{H}(P)^2)$.

The splitting step expects dual form. Using the normal formula for all steps feeds the splitter a Hadamard-transformed null point with no zero $U_{i,j}(0)$ entry, causing verification to fail after an otherwise correct 125-step chain.

A short remark in Algorithm 8.47 noting that the last two steps omit the final Hadamard (and the ultimate step pre-Hadamards the input) would prevent this.

8 Torsion basis slot convention

Affects Algorithm 2.2 (TorsionBasisFromHint), Algorithm 2.5 (ChangeOfBasis), Algorithm 4.7 (ComputeChallengeIsogeny), Algorithm 4.8 (SetChangeOfBasisMatrix), and Algorithm 4.9 (Verify). Critical

The spec says Algorithm 2.2 returns the triple $(P, Q, P{-}Q)$. The C reference stores $P{-}Q$ in the Q slot and Q in PmQ, so Ladder3pt computes $P + [m](P{-}Q)$, not $P + [m]Q$. The swap ensures the multiplied point avoids $(0, 0)$ but is not documented.

Why this is critical, not medium. The shuffled storage layout is not a local TorsionBasisFromHint quirk: it is the convention in which M_{chl} (Algorithm 4.8, signature byte 96 onwards) is encoded. Ladder3pt reads the basis triple as $(P, P{-}Q, Q)$; both signing's SetChangeOfBasisMatrix and Algorithm 4.9 line 14's $(P, Q) \leftarrow M_{\text{chl}} \cdot (P_{\text{can}}, Q_{\text{can}})$ run LadderBiscalar with that same shuffled triple. The matrix entries therefore express "coordinates of the response basis in the $(P, P{-}Q)$ basis," not in the (P, Q) basis the spec describes.

If the implementer reads the spec literally and assembles M_{chl} against $(P, Q, P{-}Q)$, the bytes still parse, verify still runs, and the resulting basis on the codomain *collapses to a singular pair* — typically $P_{\text{chl}} = Q_{\text{chl}}$ bit-for-bit, since the matrix's rows project the canonical basis onto the same point in the wrong frame. The $(2, 2)$-chain inside Algorithm 4.9 either crashes (lift_basis fails, since the lift expects a non-degenerate triple) or reaches its terminal theta null with $\#\{i, j : U_{i,j}(0) = 0\} = 0$. Verification then rejects, and the diagnosis points everywhere except at the basis layout.

The same convention must be used end-to-end: at TorsionBasisFromHint output, at every basis manipulated in SplitAuxiliaryIsogeny, ComputeEvenNonBacktrackingResponse, ComputeChallengeIsogeny, and at the change-of-basis matrix encoding and decoding. Mixing layouts at any one of those sites breaks signing/verify interoperability with itself, not just with the C reference.

Recommendation. State the layout once at the top of §2.2 as a normative convention, and reference it explicitly at every algorithm that manipulates a torsion basis triple — especially Algorithm 2.5, Algorithm 4.7, Algorithm 4.8, and Algorithm 4.9's line 14. An implementer who sees "$(P, Q, P{-}Q)$" uniformly in pseudocode has no way of knowing that Ladder3pt expects the difference in the second slot.

9 TorsionBasisFromHint fallback search must be bounded

Affects Algorithm 2.2 (TorsionBasisFromHint), and transitively verification (Algorithm 4.9). High

Algorithm 2.2 reconstructs a 2-torsion basis from a curve coefficient A and a 7-bit hint h. When $h = 0$ (the rare case where the smallest admissible scalar n does not fit in 7 bits), the algorithm falls back to a search: starting at $n = 128$, increment until the predicate $\big(nA$ on E_A and not a square$\big)$ holds (or the dual predicate, when A is a residue). The pseudocode reads "while not satisfied: $n \leftarrow n + 1$," with no upper bound on n.

What goes wrong. TorsionBasisFromHint is invoked from Algorithm 4.9 (Verify) on three different curves (E_{pk}, E_{aux}, E_{chl}) whose coefficients are reconstructed directly from signature bytes. An adversary who controls those bytes can supply a curve coefficient A for which no n in the entire search window satisfies the predicate. A literal implementation of the spec pseudocode loops forever; verification never returns.

This is a denial-of-service against `verify`, reachable from any unauthenticated input. We discovered it via the Wycheproof SQIsign suite, which contains a regression vector (`HintOverflow`

flag) for exactly this case. A direct spec-to-code translation hangs on that vector regardless of the implementation language.

Recommendation. Algorithm 2.2 should state explicitly that the fallback search is bounded: "stop after at most K iterations and signal failure to the caller; on signal, Verify rejects the signature." A constant of $K = 2^{16}$ provides several orders of magnitude of headroom above anything observed on honest signatures (where the admissible n uniformly fits in 7 bits) and is small enough that the verifier remains $O(1)$ on adversarial input.

Algorithm 4.9 (Verify) should be amended to consume the new failure signal from TorsionBasisFromHint as a rejection. The pattern is the same one already established in §J (GetIndexSplitting's "uniqueness" assumption): *the spec's correctness guarantees apply to honest inputs; verify must impose an explicit cap and a rejection path on adversarial ones.*

10 Matrix storage convention for M_{chl} and M_{sk}

Affects Algorithm 2.5 (ChangeOfBasis), Algorithm 4.1 (KeyGen: M_{sk}), Algorithm 4.8 (SetChangeOfBasisMatrix: M_{chl}), and Algorithm 4.9 (Verify: line 14 application of M_{chl}). Critical

The pseudocode of Algorithm 2.5 returns "x_1, x_2, x_3, x_4 such that $Q_1 = [x_1]P_1 + [x_2]P_2$ and $Q_2 = [x_3]P_1 + [x_4]P_2$." The spec does not say how to lay these four scalars out as a 2×2 matrix on the wire (§4.6 simply concatenates the four components). Two natural layouts produce the same wire bytes only if the matrix is written in column-major form $\left[\begin{smallmatrix} x_1 & x_3 \\ x_2 & x_4 \end{smallmatrix}\right]$, which is what the C reference does in `change_of_basis_matrix_tate` and consumes via `matrix_application_even_basis` (column 0 yields the new P). An implementer who reads "$Q_1 = [x_1]P_1 + [x_2]P_2$" as a *row* ($x_1, x_2$ on row 0, x_3, x_4 on row 1) writes *transposed* bytes: positions for x_2 and x_3 get swapped.

What goes wrong. The transposed encoding is not detectable from a single signature because parsing succeeds, the matrix entries are still bounded, and Algorithm 4.9 runs. The defect manifests only after Algorithm 4.9 line 14 applies M_{chl} to the canonical basis: instead of $(P_{\text{chl}}, Q_{\text{chl}})$ the result is $([x_1]P + [x_3]Q, [x_2]P + [x_4]Q)$, the application of M_{chl}^T. The $(2,2)$-chain that follows then proceeds on the wrong torsion basis, and verification rejects with no useful signal. M_{sk} has the same issue inside Algorithm 4.1.

This is a different bug from the basis-slot swap of §6. Even with the basis layout fixed, the matrix-encoding ambiguity remains and produces identical-looking failures.

Recommendation. Either:

1. State Algorithm 4.8 explicitly produces the matrix $\left[\begin{smallmatrix} x_1 & x_3 \\ x_2 & x_4 \end{smallmatrix}\right]$ (column-major, with column 0 the coordinates of Q_1 and column 1 the coordinates of Q_2), and Algorithm 4.9 line 14 applies it by reading column 0 to get the first new basis point; or
2. Re-state the wire encoding in §4.6 in basis-coordinate terms: *"$M_{\text{chl}}[[0]]$ are the bytes of x_1; $M_{\text{chl}}[[1]]$ of x_3; $M_{\text{chl}}[[2]]$ of x_2; $M_{\text{chl}}[[3]]$ of x_4,"* so the transpose ambiguity cannot survive a careful reading.

11 Spec's 2^{f-e} scaling in Algorithm 2.5 is wrong for Algorithm 4.8

Affects Algorithm 2.5 (ChangeOfBasis) and Algorithm 4.8 (SetChangeOfBasisMatrix). Critical

Line 4 of Algorithm 2.5 returns $x_i \leftarrow 2^{f-e} \cdot \log_{\zeta}(\zeta_i)$. The 2^{f-e} factor scales the e-bit dlog up to a 2^f-precision value, so the matrix can be applied to a full 2^f-torsion basis.

But Algorithm 4.8 (lines 12–13) reduces both bases to order 2^e *before* computing the matrix. Once the bases are reduced, the 2^{f-e} scaling is wrong: applying an f-bit entry to a 2^e-torsion point ignores everything above the e-bit window. The wire encoding (§4.6) stores $\lceil e/8 \rceil$ bytes per entry; the spec's shift puts the dlog above this window, so the encoded bytes are zero except for a few bits at the boundary.

The C reference's `_change_of_basis_matrix_tate` omits the 2^{f-e} scaling and stores raw dlogs. This is undocumented but necessary for the wire format and reduced-basis application to work.

An implementer following Algorithm 2.5's step 4 literally produces signatures whose M_{chl} entries are mostly zero on the wire. Verification accepts and runs but Algorithm 4.9's line 24 chain collapses to all-zero theta nulls; nothing flags "M_{chl} entries unexpectedly small."

Recommendation. Either fold the basis reduction into Algorithm 2.5, or rewrite Algorithm 2.5 step 4 conditionally: "return $\log_\zeta(\zeta_i)$ when the bases are pre-reduced to order 2^e; return $2^{f-e} \cdot \log_\zeta(\zeta_i)$ when applied to a full 2^f-torsion basis." As written, the algorithm is correct for its declared semantics (full-torsion application) but wrong for how every protocol algorithm actually uses it.

12 Cubical Tate third-argument sign convention in Algorithm 2.5

Affects Algorithm 2.5 (ChangeOfBasis) step 1, and transitively every algorithm whose change-of-basis matrix is recovered by cubical Tate pairings (Algorithm 4.8's M_{chl} inversion). CRITICAL

The cubical Tate pairing $t_{2^e}(P, Q, R)$ takes a third projective argument R that fixes the sign branch of the differential addition ladder. The values $t_{2^e}(P, Q, P{+}Q)$ and $t_{2^e}(P, Q, P{-}Q)$ are multiplicative inverses of each other:

$$t_{2^e}(P, Q, P{-}Q) \;=\; t_{2^e}(P, Q, P{+}Q)^{-1}.$$

Both satisfy bilinearity in their respective convention, both test as 2^e-th roots of unity, and the two conventions are individually self-consistent.

Algorithm 2.5 step 1 writes $\zeta \leftarrow t_{2^e}(P_1, P_2)$ without naming the third argument's sign. Step 3 then computes $\zeta_i \leftarrow t_{2^e}(R_i, P_j, R_i \pm P_j)$ for each target basis point R_i against the canonical pair (P_1, P_2).

What goes wrong. An implementer who picks the SUM convention ($R = P_1{+}P_2$) for the four cross-pairings ζ_i but DIFFERENCE ($R = P_1{-}P_2$, e.g. because the basis triple already carries $P_1{-}P_2$ in the third slot per §6) for the base ζ silently inverts the relationship between ζ and the ζ_i: the base becomes T^{-1} where the cross-pairings remain T_i, and $\log_\zeta(\zeta_i)$ returns $-x_i \bmod 2^e$ instead of x_i.

For a target basis with all coefficients in $\{1, 2\}$ (the typical case in Algorithm 4.8), every dlog comes out near $2^e - 1$ or $2^e - 2$, so every matrix cell ends with its low limbs entirely set to `0xff...ff` with at most a few low bits differing. Verification accepts the matrix syntactically (entries within bounds), applies it to the canonical basis, and obtains a *non-degenerate but wrong* basis. The $(2, 2)$-chain then runs to completion, lands on a non-product theta null, and verify rejects. Without intermediate diagnostics the symptom looks like "the chain is just wrong" rather than a sign-flip in the pairing setup.

The bug is invisible to most unit tests because each individual convention is internally consistent. Bilinearity-in-the-first test with SUM holds. Bilinearity-in-the-first test with DIFFERENCE holds. Antisymmetry holds. Round-trip FROM_BASES test with a known matrix is the only shape that fires — and even then only when the test uses small positive matrix entries (small negatives masquerade as near-2^e values and the symptom is washed out).

Recommendation. State the third-argument convention explicitly in Algorithm 2.5. The natural choice is the SUM convention everywhere (consistent with Jacobian addition's natural output), in which case step 1 should read $\zeta \leftarrow t_{2^e}(P_1, P_2, P_1{+}P_2)$. Equivalently, an implementer who reads Algorithm 8.12 (DifferencePoint) and naturally has $P_1{-}P_2$ available must compute $P_1{+}P_2$ separately for step 1 rather than reusing the available difference. Either convention is mathematically fine; mixing them is catastrophic.

13 Matrix exponent in Algorithm 4.8 omits `HD_extra_torsion`

Affects Algorithm 4.8 (SetChangeOfBasisMatrix) and Algorithm 4.9 (Verify). High

Algorithm 4.8's pseudocode (lines 12–13) describes scaling the bases by $2^{f-e'_{\text{rsp}}-r_{\text{rsp}}}$, leaving them at order $2^{e'_{\text{rsp}}+r_{\text{rsp}}}$, and computing the matrix at this exponent. The dlog at this exponent recovers the low $e'_{\text{rsp}} + r_{\text{rsp}}$ bits of each scalar relationship; the high bits are silently zero.

But Algorithm 4.9's line 24 $(2,2)$-chain consumes `HD_extra_torsion` $= 2$ bits of additional torsion: it needs the basis at order $2^{e'_{\text{rsp}}+r_{\text{rsp}}+2}$. The C reference does both: it reduces to $2^{e'_{\text{rsp}}+r_{\text{rsp}}+2}$ *and* computes the matrix at the same exponent. The two extra bits in the matrix entries are essential for verify to reproduce the basis the chain needs.

The spec's Algorithm 4.8 omits the +2 silently. An implementer who follows the pseudocode exactly produces a matrix whose entries truncate two high bits. Verify then runs but its post-matrix basis is wrong in those two bits, and the chain fails with all-zero theta nulls — same symptom as the previous two findings, indistinguishable from those without inspecting matrix entries directly.

Recommendation. Add "+`HD_extra_torsion`" to the exponents in Algorithm 4.8 lines 11 and 13, matching the C reference's `reduced_order = pow_dim2_deg_resp + HD_extra_torsion + two_resp_length`. This makes the dependency on Algorithm 4.9's torsion needs explicit at the matrix-construction site, where it can actually be implemented.

14 Normalization of $(A_{24} : C_{24})$

Affects Algorithm 2.2 (TorsionBasisFromHint). Medium

The C reference normalizes to $((A{+}2)/4 : 1)$ before basis generation. If the curve arrives from an isogeny codomain with $C_{24} \neq 1$, the Montgomery ladder produces a different projective representative, which propagates through DifferencePoint and Ladder3pt.

Note in Algorithm 2.2 that the curve must be in normalized form before computing the basis.

15 Gluing requires Jacobian y-coordinates

Affects Algorithm 8.39 (GluingEval). Low

Montgomery x-only arithmetic cannot distinguish $P + Q$ from $P - Q$. The gluing evaluation needs this distinction. The C reference lifts to Jacobian (x, y, z) via Okeya–Sakurai for this step—the only place in verification that needs y-coordinates. Note this in Algorithm 8.39.

16 Jacobian doubling for balanced strategy

Affects Algorithm 8.47, Phase 1. Low

The spec does not specify whether Phase 1 doublings use Montgomery or Jacobian coordinates. The C reference doubles in Jacobian, then converts via `jac_to_xz`: $(x, y, z) \mapsto (x, z^2)$. The theta

coordinate computations are sensitive to the projective representative. Note this, or at minimum state that the representative matters.

17 DifferencePoint normalization factor

Affects Algorithm 8.12 (DifferencePoint). Low

The C reference multiplies B_{XX}, B_{XZ}, B_{ZZ} by $C \cdot \overline{C}^2 \cdot \overline{Z_P}^2 \cdot \overline{Z_Q}^2$ (Galois conjugation) before the quadratic solve. This makes the discriminant a fourth power in $\mathbb{F}_p$, reducing the $\mathbb{F}_{p^2}$ sqrt to an $\mathbb{F}_p$ sqrt. We initially used $(Z_P Z_Q)^2$, which lacks this property since $\overline{z}^2 \neq z^2$ in general. State the normalization factor and its purpose in Algorithm 8.12.

References

[1] SQIsign team, *SQIsign: Compact Post-Quantum Signatures from Quaternions and Isogenies*, Specification version 2.0.1, dated 2025-07-07, https://sqisign.org/spec/sqisign-20250707.pdf.

[2] M. N. H. Aardal, M. P. Azimova, E. L. Carrión, L. Dillinger, and M. J. Kannwischer, *Optimized One-Dimensional SQIsign Verification on Intel and Cortex-M4*, Cryptology ePrint Archive, Paper 2024/1563, https://eprint.iacr.org/2024/1563.

[3] C. Costello, H. Hisil, and B. Smith, *Faster Compact Diffie–Hellman: Endomorphisms on the x-line*, ASIACRYPT 2017, https://eprint.iacr.org/2017/518.

A Never recompute $P-Q$ via DifferencePoint

Affects Algorithm 4.9 (verification), lines 11–26, and transitively the $(2,2)$-isogeny chain. Critical

A torsion basis $(P,\ Q,\ P-Q)$ is produced by TorsionBasisFromHint (Algorithm 2.2). The third component $P-Q$ is computed once, via DifferencePoint, which internally takes an $\mathbb{F}_{p^2}$ square root. Subsequent operations—scaling (repeated doubling), matrix application (Algorithm 4.9 line 14), and the even response isogeny (line 17)—must propagate all three points together. If at any point $P-Q$ is recomputed from P and Q via DifferencePoint, the fresh square root may pick the opposite branch, producing a point that is *affinely distinct* from the original $P-Q$.

Concretely, the two branches of DifferencePoint yield $x(P-Q)$ and $x(P+Q) = x(2P-Q)$; these are different points. All subsequent differential additions (the three-point ladder, the biladder for matrix application, and the $(2,2)$-isogeny chain's Ladder3pt and lift_basis) consume $P-Q$ as a third input and produce wrong results if it is silently replaced by $2P-Q$.

In our implementation, three separate recomputations caused failures:

1. After scaling the basis by repeated doubling, we reconstructed $P-Q$ from the doubled P and Q instead of doubling $P-Q$ alongside them.
2. The matrix application $M_{\text{chl}} \cdot (P,Q)$ computed $P' - Q'$ via DifferencePoint(P',Q') instead of a third biladder call $[(a-b)]P + [(c-d)]Q$.
3. Before entering the $(2,2)$-chain, we recomputed $P-Q$ from the post-isogeny images instead of pushing the original $P-Q$ through the isogeny as a third evaluation point.

Each of these independently caused KAT verification failure. Fixing all three was necessary and sufficient to make KAT vector 0 pass.

Current spec text. The spec does not warn against recomputing $P-Q$. Algorithms that scale or transform a basis mention only P and Q; the third point $P-Q$ is not discussed.

Recommendation. Add a normative note in §2.2 or §8.2:

> *Once a basis $(P,Q,P-Q)$ has been produced by TorsionBasisFromHint, the third point $P-Q$ must be propagated through every subsequent operation (doubling, matrix application, isogeny evaluation) alongside P and Q. Recomputing $P-Q$ via DifferencePoint is not permitted: the square root may select a different branch, yielding an affinely distinct point that silently corrupts all downstream differential additions.*

B Change-of-basis matrix application convention

Affects Algorithm 4.8 (SetChangeOfBasisMatrix), Algorithm 4.9 (verification, line 14). High

The spec does not specify whether the change-of-basis matrix $\mathbf{M}_{\text{chl}}$ is applied to a torsion basis (P,Q) by rows or by columns.

What the spec says. Algorithm 4.8 line 6 writes $(P_2,Q_2) \leftarrow \mathbf{M}_1 \cdot (P_2,Q_2)$ without specifying the convention. Algorithm 4.9 line 14 writes $(P_{\text{chl}},Q_{\text{chl}}) \leftarrow \mathbf{M}_{\text{chl}} \cdot (P_{\text{chl}},Q_{\text{chl}})$, again without specifying.

What the C reference does. `matrix_scalar_application_even_basis` in `verify.c` applies the matrix by *columns*:

$$R' = [a]\,P + [c]\,Q \qquad \text{(column 0: } \texttt{mat[0][0]}, \texttt{mat[1][0]}\text{)}$$
$$S' = [b]\,P + [d]\,Q \qquad \text{(column 1: } \texttt{mat[0][1]}, \texttt{mat[1][1]}\text{)}$$

where $\mathbf{M} = \left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$. An implementor who applies by rows (the natural reading of "$\mathbf{M} \cdot (P,Q)^T$") will produce wrong results.

Impact. Applying by rows instead of columns silently produces the wrong torsion basis, causing the (2,2)-isogeny chain in verification to fail (typically a panic in the splitting step).

Recommendation. State explicitly in Algorithm 4.8 and Algorithm 4.9 that the matrix is applied by columns, or equivalently define the multiplication as $(P', Q') = (P, Q) \cdot \mathbf{M}^T$.

C RandomIdealGivenNorm: β sampling range is inclusive of N

Affects Algorithm 3.10 (RandomIdealGivenNorm) during signing's auxiliary-ideal construction (Algorithm 4.2 line 16). High

Algorithm 3.10 step 11 reads "sample $\beta = x + yi + zj + wk$ with $x, y, z, w \in [1, N]$ and $\gcd(\mathrm{nrd}(\beta), N) = 1$." The bracket notation is ambiguous on the endpoint: an implementor can read $[1, N]$ as $\{1, 2, \ldots, N\}$ (closed) or as $\{1, 2, \ldots, N-1\}$ (half-open, "mod N" style). Both distributions are uniform over their respective ranges and both preserve the algorithm's correctness as a math object.

Why the choice matters for byte-interop. On a shared DRBG the two interpretations consume the same number of bytes per attempt (rejection sampling on a top-byte mask), but the *value* accepted differs by 1: under $[1, N]$ a sampled hex-bits-equal-zero case maps to 1 (the addition of the left endpoint), while under $[1, N-1]$ the same bytes are rejected. For every other byte pattern the two accepted values differ by exactly 1. A constant shift of each β coordinate by 1 propagates through $\alpha = \gamma\beta$ (a quaternion product) as $\alpha_{\text{ours}} = \alpha_{\text{ref}} - \gamma(1 + i + j + k)$, which is generally not in $O_0 \cdot N$. The resulting left ideal $O_0\alpha + O_0 N$ is therefore a different $\mathbb{Z}$-module on the two sides, not just a different basis representation.

The downstream sensitivity is severe: signing's response phase intersects this ideal with $I_{\text{com,rsp}}$, then feeds the intersection through SuitableIdeals. A different Z-module gives a different L^2-reduced basis, a different choice of (s, t, β_s, β_t), a different α_{rsp}, and a different signature on the wire.

What the C reference does. `ibz_rand_interval(out, 1, norm)` samples uniformly on $\{1, 2, \ldots, N\}$ inclusive of N, by reading $\lceil \log_2 N/8 \rceil$ bytes, masking to $\lceil \log_2(N-1) \rceil$ bits, rejecting samples $> N-1$, and returning tmp + 1. This is the closed-interval interpretation.

Recommendation. Algorithm 3.10 should state the convention explicitly — the notation should be either "$[1, N]$ inclusive" with a forward pointer to "`ibz_rand_interval`"-style sampling, or "$[1, N-1]$" (half-open) and a corresponding alternative sampling primitive. Either convention is fine cryptographically; mixing them across a reference implementation and a derivative implementation breaks KAT byte-equality and produces signatures that fail interoperability checks.

The same ambiguity applies to every other "sample uniformly in $[a, b]$" statement in the spec. A short normative remark near the top of §3 stating that all such ranges are closed and that the canonical sampler is "tmp + a with tmp $\in [0, b-a]$ by top-byte masked rejection" would prevent the entire class of off-by-one mismatches.

D GeneralizedRepresentInteger sampling ranges and divisibility

Affects Algorithm 3.12 (GeneralizedRepresentInteger), and transitively Algorithm 3.15 (FixedDegreeIsogeny) during signing. Medium

Algorithm 3.12 describes sampling z from $[1, \ldots, m]$ (line 4) and t from $[-m', \ldots, m']$ (line 5). Two aspects of the algorithm are easy to implement incorrectly:

1. Sign of t matters for the isogeny condition. Since $M' = 4M - p(z^2 + qt^2)$ depends on t^2, it is tempting to sample only $t \geq 0$ and rely on the symmetry. However, the isogeny condition (line 15) checks $x - t \equiv 2 \pmod 4$, which *does* depend on the sign of t. Restricting to $t \geq 0$ halves the effective search space for the isogeny condition, which can cause the algorithm to exhaust its bound without finding a solution.

2. Divisibility by 2 in $\mathcal{O}$ (line 18) is not coordinate-wise. Line 18 says "set d to be the biggest scalar such that $\gamma/d \in \mathcal{O}$" and line 19 checks $d = 2$. For the standard order $\mathcal{O}_0 = \mathbb{Z}[i] + j\,\mathbb{Z}[i]$ with $q = 1$ and $\omega = i$, an element $\gamma = x + iy + jz + kt$ satisfies $\gamma/2 \in \mathcal{O}_0$ if and only if x, y, z, t are all even. However, for orders with half-integer basis elements (e.g., $\mathcal{O}_0 = \mathbb{Z} + i\mathbb{Z} + \frac{1+j}{2}\mathbb{Z} + \frac{i+k}{2}\mathbb{Z}$, which is the actual $\mathcal{O}_0$ for SQIsign), divisibility by 2 in $\mathcal{O}_0$ is *not* equivalent to all coordinates being even in the $\{1, i, j, k\}$ basis. The correct check must account for the order's common denominator and basis structure.

An implementor who reads "$\gamma/d \in \mathcal{O}$" as "all coordinates divisible by d" (which is correct for $\mathbb{Z}^4$ but not for a non-trivial order) will write code that rejects valid solutions.

Recommendation. Add a remark after line 18 of Algorithm 3.12 clarifying:

> *The divisibility check "$\gamma/d \in \mathcal{O}$" is with respect to the order $\mathcal{O}$, not coordinate-wise in the $\{1, i, j, k\}$ basis. For orders with a non-trivial denominator $d_{\mathcal{O}}$, the integral coordinates of γ in the order's column basis must all be divisible by $d \cdot d_{\mathcal{O}}$.*

Also note explicitly that the range $[-m', m']$ on line 5 includes negative values, and that both signs must be tried to satisfy the isogeny condition on line 15.

3. Product order in γ construction (line 17). Line 17 writes $\gamma \leftarrow (x + \omega y + jz + \omega jt)$. In quaternion algebra, $\omega j \neq j\omega$ in general ($ij = k$ but $ji = -k$). The C reference's `quat_order_elem_create` computes $j \cdot \omega \cdot t$ (i.e., left-multiplying by j then by ω), which for $\omega = i$ gives $ji \cdot t = -kt$. Reading line 17 as $\omega jt = ij \cdot t = kt$ produces the opposite sign on the k component.

The difference does not affect the norm (nrd depends only on $|\text{components}|^2$), but it *does* affect the isogeny condition (line 15) and the divisibility check (line 18), since both depend on the parity and congruence class of the coordinates.

State whether line 17 means $(\omega j)t$ or $(j\omega)t$, or equivalently state the product order used by `quat_order_elem_create`.

E Response lattice: product vs. intersection

Affects Algorithm 4.2 (SQIsign.Sign), line 14. HIGH

Algorithm 4.2 line 14 writes

$$\alpha_{\text{rsp}} \leftarrow \textsc{RandomEquivalentQuaternion}(\overline{I_{\text{com}}} \cap I_{\text{sk}} \cdot I_{\text{chl}})$$

The notation "$I_{\text{sk}} \cdot I_{\text{chl}}$" is ambiguous: it could mean the *ideal product* (the $\mathbb{Z}$-span of all $\alpha\beta$ with $\alpha \in I_{\text{sk}}$, $\beta \in I_{\text{chl}}$) or the *intersection* $I_{\text{sk}} \cap I_{\text{chl}}$.

The C reference (`sign.c:81–85`) computes:

1. $I_{\text{chl,secret}} = I'_{\text{chl}} \cap I_{\text{sk}}$ (intersection, **not** product)
2. $\overline{I_{\text{com}}}$ (conjugate of I_{com})

3. $\alpha_{\text{rsp}} \leftarrow \textsc{Sample}(I_{\text{chl,secret}} \cap \overline{I_{\text{com}}})$

Furthermore, the C ref intersects the *raw* challenge ideal I'_{chl} (before pushforward) with I_{sk}, not the pushed-forward $I_{\text{chl}} = [I_{\text{sk}}]_* I'_{\text{chl}}$.

An implementor who reads "$I_{\text{sk}} \cdot I_{\text{chl}}$" as an ideal product will compute a completely different lattice. The ideal product's norm is $\text{nrd}(I_{\text{sk}}) \cdot \text{nrd}(I_{\text{chl}}) \approx 2^{381}$, while the intersection's norm divides $\text{lcm}(\text{nrd}(I_{\text{sk}}), \text{nrd}(I_{\text{chl}}))$. Sampling from the wrong lattice produces elements that generate trivial ideals (the HNF collapses to a scaled copy of O_0), causing the response phase to fail silently.

Recommendation. Replace "$I_{\text{sk}} \cdot I_{\text{chl}}$" with explicit "$I_{\text{sk}} \cap I'_{\text{chl}}$" and note that the intersection uses the pre-pushforward challenge ideal.

F LatticeSampling algorithm is underspecified

Affects Algorithm 3.3 (LatticeSampling), used by Algorithm 4.3 (RandomEquivalentQuaternion) which is the inner sampling of Algorithm 4.2 (SQIsign.Sign) line 14. Medium

Algorithm 3.3 is described as "sample uniformly from the lattice L elements of reduced norm at most ρ" without specifying the algorithm. An implementor's natural reading–LLL-reduce the primal Gram of L, set per-axis bounds $\text{box}[i] = \sqrt{\rho / G_{\text{red}}[i][i]}$, and reject sample–is correct but produces a bounding parallelogram much looser than the input ellipsoid, with a typical acceptance rate around 10^{-4} at the radii used in signing.

The C reference's `quat_lattice_sample_from_ball` (`lat_ball.c:58`) implements a different, tighter algorithm: LLL-reduce the *dual* Gram $\text{adj}(G)$ while tracking the unimodular transformation U, set $\text{box}[i] = \sqrt{\text{dual}G_{\text{red}}[i][i] \cdot \rho / \det G}$, sample y in the box, map $x = U^{-\top} y$, and check $x^\top G x \le \rho$ on the original primal Gram. This bound corresponds to the ellipsoid's inertia tensor (via the dual lattice's reduction) and gives an acceptance rate within a small Hermite-constant factor of optimal.

The two algorithms produce different distributions of accepted α, and downstream Kani-kernel acceptance in Algorithm 4.5 (SplitAuxiliaryIsogeny) tracks the α distribution. An implementor following the spec literally will produce a different per-iter retry profile than the C ref and may attribute the timing or convergence differences to other algorithms entirely. We did, until cross-checking the C ref's sampling code revealed the gap.

Recommendation. Spell out the dual-LLL procedure (and the $\text{box}[i] = \sqrt{\text{dual}G_{\text{red}}[i][i] \cdot \rho / \det G}$ bound, plus the $U^{-\top}$ map) in Algorithm 3.3, or at least name it as the recommended algorithm with a citation to a standard reference. As stated, the spec permits any sampling that meets the distribution, but the C reference's choice is the load-bearing one for matching its benchmark timings.

G L^2-deep-insertion iteration order underspecified

Affects Algorithm 3.6 (L^2-DeepInsertion), used by every LLL reduction in keygen and signing through `quat_lideal_prime_norm_reduced_equivalent`. High

Algorithm 3.6 defines the deep-insertion point as "the smallest j such that $\bar{\delta} \cdot r[j][j] > t[j]$," i.e. the smallest level at which the L^2-condition fails on the partial squared-norm sequence $t[0], \ldots, t[\kappa - 1]$.

The C reference's `quat_lll_core` (`quaternion/ref/generic/lll/l2.c:110-114`) uses a different iteration order:

```
for (swap = kappa; swap > 0; swap--) {
    if (delta_bar * r[swap-1][swap-1] < lovasz[swap-1])
```

```
        break;
}
```

This iterates s *from* κ *down to* 1 and breaks on the first level where the L^2-condition still holds.

When the failure pattern is monotonic—fails for $j < a$, holds for $j \geq a$—both formulations return $s = a$. But the partial squared norms $t[j]$ are computed from a running Gram–Schmidt orthogonalization update on floating-point scalars (`dpe_t` in the C ref). Under rounding the failure pattern can be *non-monotonic*: failures at, say, $j = 0$ and $j = 2$ but a hold at $j = 1$. The spec-literal form returns $s = 0$ (smallest failing); the C ref returns $s = 2$ (largest level above the deepest contiguous hold). Both are valid L^2-reduced bases, but the resulting bases differ in the sign of column zero, propagating to off-diagonal sign flips in the post-LLL Gram matrix ($G[0][2]$, $G[0][3]$ and their transposes; diagonals match byte-for-byte).

The downstream call to `quat_lideal_prime_norm_reduced_equivalent` (`lll_applications.c`) rejection-samples short vectors c and accepts the first c whose class quadratic form $c^\top G_{\text{class}} c$ is prime. A different post-LLL G_{class} produces a different acceptance, hence a different equivalent prime ideal, hence a different chain codomain, hence a different public key.

Empirical impact. For 16 of 100 NIST-I keygen KATs every input that hit the non-monotonic condition—this single iteration-order divergence cascaded all the way to a public-key mismatch. Switching from the spec-literal order (smallest failing j, ascending) to the C ref order (deepest contiguous failure, descending) moved our keygen byte-equality count from 83/100 to 99/100 in a single commit.

Current spec text. Algorithm 3.6 (and Algorithm 3.3 which calls it) describe the deep-insertion choice mathematically but do not specify the iteration direction. Both orders satisfy the spec's English description under the implicit monotonicity assumption; floating- point rounding can break that assumption silently.

Recommendation. Either (a) specify the iteration order in Algorithm 3.6—"iterate $s = \kappa, \kappa{-}1, \ldots, 1$ and stop at the first s such that $\bar{\delta} \cdot r[s-1][s-1] \geq t[s-1]$"—or (b) note that the choice of order is implementation-defined but that KAT byte-equality requires matching the reference's order exactly. Without this clarification, an implementor who follows the spec's "smallest j that fails" will produce a non-interoperable build and need to read `quat_lll_core` source to find the gap.

H FixedDegreeIsogeny missing u^{-1} normalization

Affects Algorithm 3.15 (FixedDegreeIsogeny), and transitively Algorithm 3.13 (IdealToIsogeny) during signing. Critical

Algorithm 3.15 line 2 computes $\theta \leftarrow \text{RepresentInteger}(u \cdot (2^e - u), O_t, \text{true})$, and line 4 constructs the $(2,2)$-isogeny kernel from $([u]P,\ \theta(P))$ and $([u]Q,\ \theta(Q))$. Since $\text{nrd}(\theta) = u \cdot (2^e - u)$, the endomorphism θ has degree $u(2^e - u)$; the kernel we want is that of the degree-$(2^e - u)$ isogeny, generated by $(P,\ (\theta/u)(P))$.

The C reference multiplies all four quaternion coordinates of θ by $u^{-1} \bmod 2^{e+2}$ before applying the endomorphism to the torsion basis (`dim2id2iso.c`, lines 133–143). Without this normalization, the ActionByTranslation determinant of the gluing kernel is degenerate: the splitting step finds zero (or ten) vanishing $U_{i,j}(0)$ indices instead of exactly one, and the $(2,2)$-chain fails.

Current spec text. Algorithm 3.15 line 4 writes "$K_1 = (u \cdot P_t,\ \theta(P_t))$" without mentioning the u^{-1} normalization. The pseudocode is algebraically suggestive (the u-torsion is killed after $f{-}2{-}e$

doublings), but in practice the non-normalized kernel is numerically degenerate and no $(2,2)$-chain succeeds.

Recommendation. Replace line 4 of Algorithm 3.15 with $(P_t,\ (\theta \cdot u^{-1} \bmod 2^{e+2})(P_t))$ and note that u is odd (hence invertible mod 2^{e+2}) and that the extra +2 in the modulus accounts for the gluing's order-4 structure.

I Fixed-precision Hermite Normal Form

Affects Algorithm 3.2 (HNF), and transitively every algorithm that constructs or reduces an ideal lattice — including Algorithm 3.9 (RandomEquivalentPrimeIdeal), Algorithm 3.10 (RandomIdealGivenNorm), and Algorithm 3.15 (FixedDegreeIsogeny). High

Algorithm 3.2 describes the column-style Hermite Normal Form via extended-GCD elimination. The pseudocode operates over $\mathbb{Z}$, i.e., arbitrary-precision integers.

What goes wrong. Classical HNF's intermediate entries can grow exponentially in the rank. For rank-4 quaternion ideal lattices at NIST-I the initial column entries can reach $\sim 2^{770}$ bits (the $p \cdot g_i$ products from Algorithm 3.10). After a few xgcd elimination steps, intermediate column entries exceed any reasonable fixed-width budget. At our storage width of 1920 bits (30×64), the resulting "HNF" silently wraps: we observed basis columns collapse to elements of the ambient order O_0 (reduced norms of 4 and $\sim 2^{251}$) rather than the intended ideal I. Every subsequent operation — RandomEquivalentPrimeIdeal, IdealToIsogeny, the entire response phase — then operates on a lattice unrelated to I.

What the C reference does. The C reference uses GMP (`ibz_t`), so coefficient growth is absorbed by arbitrary-precision arithmetic. The classical algorithm works correctly there.

Fix. Use the standard Domich–Kannan–Trotter modular HNF (Cohen, *A Course in Computational Algebraic Number Theory*, §2.4.2): given a positive integer D with $D \cdot e_i \in L$ for every standard basis vector e_i, reduce every intermediate entry modulo D after every arithmetic update. This bounds entries by D instead of by the exponential classical bound, at the cost of computing at a wider intermediate width $W \geq 2 \cdot \lceil \log_2 D \rceil / 64$. The result is identical to the classical HNF in $\mathbb{Z}$.

The modulus must be a lattice member, not just a covolume multiple. The Cohen 2.4.8 construction implicitly takes $L \cup D \cdot \mathbb{Z}^n$ and returns its HNF, which equals $\mathrm{HNF}(L)$ exactly when $D \cdot \mathbb{Z}^n \subseteq L$. In particular, for left ideals $I \subseteq O_0$, the reduced norm $N(I)$ has $N(I) \cdot O_0 \subseteq I$ and is therefore admissible; arbitrary multiples of $\det(L)$ need not satisfy $D \cdot e_i \in L$ and produce a strictly coarser super-lattice.

An implementor who reads "D is a multiple of $\det(L)$" without the membership condition can derive a closed-form covolume formula like $D = c \cdot d^4 \cdot N^2 \cdot p$ for some scalar c, hand that to the modular HNF, and silently construct $L + D \cdot \mathbb{Z}^n$ instead of L. In the SuitableIdeals cross-order step ($I^{(t)} = \mathrm{conj}(I_{\mathrm{reduced}}) \cdot J_t$), the super-lattice may contain integer scalars like $(2,0,0,0)$ that should not be in $I^{(t)}$. Those scalars then enter the L^2-reduced class Gram with a zero diagonal, and the Lovász/swap loop fails to terminate — on both our implementation and the C reference's `quat_lll_core`, which we confirmed by extracting the degenerate Gram and feeding it through C-ref directly.

The C reference avoids this by computing $D := |\det(\text{first 4 product generators})|$ at runtime (`lattice.c:231-233`), which is the covolume of those generators and therefore lies in the lattice they span. `Lattice::product` (without an explicit modulus) implements the same recipe in Rust.

A precomputed covolume bound is correct only when the implementer can prove $D \cdot e_i \in L$ for the input shape — a property that covolume formulas do not capture.

For NIST-I commitment ideals, $D = 4d^4N^2p \approx 2^{1282}$ is a compile-time constant. For response-phase and post-reduce ideals the modulus varies and is computed per call.

Recommendation. Add a remark after Algorithm 3.2 noting that fixed-precision implementations must account for coefficient growth, and that the standard modular HNF technique works only when the modulus D satisfies $D \cdot \mathbb{Z}^n \subseteq L$ — not merely that D divides $\det(L)$. State the safe defaults explicitly: either $D := N(I)$ (always admissible for an ideal of norm $N(I)$) or $D := |\det(\text{first } n \text{ generators})|$ (the C reference's choice for lattice products).

J Endomorphism image $P-Q$ and the Okeya-Sakurai lift

Affects Algorithm 3.15 (FixedDegreeIsogeny), and transitively IdealToIsogeny and KeyGen. High

Algorithm 3.15 line 4 says to construct the kernel from $(\theta(P),\ \theta(Q))$. The $(2,2)$-isogeny chain lifts these Montgomery x-only points to Jacobian (x, y, z) coordinates via the Okeya-Sakurai formula, which requires the projective x-only representative of $P-Q$ (not just its affine x-coordinate).

The Okeya-Sakurai formula recovers y_Q from $(x_P, y_P, X_Q, Z_Q, X_{P-Q}, Z_{P-Q})$. The specific projective representative $(X_{P-Q} : Z_{P-Q})$ determines the recovered y_Q. Two representatives that agree in the ratio X/Z but differ in the individual coordinates produce different y_Q values—one correct, one not.

The spec does not specify how $\theta(P-Q)$ is obtained. Two natural approaches:

1. Compute $\theta(P-Q)$ as a third biladder call with scalars $(m_{00}-m_{01},\ m_{10}-m_{11})$ on the same basis triple $(P, Q, P-Q)$.
2. Compute DifferencePoint$(\theta(P),\ \theta(Q))$, which takes an $\mathbb{F}_{p^2}$ square root and is ambiguous ($P-Q$ vs. $P+Q$).

Approach (1) produces three outputs whose projective representatives are consistent (all derived from the same basis via differential addition), so the Okeya-Sakurai lift recovers the correct y. Approach (2) picks a deterministic but potentially wrong branch of the square root, causing the lift to recover the wrong y for roughly half of all inputs. In our implementation, approach (2) caused the $(2,2)$-chain to fail for every θ from RepresentInteger while succeeding for the i automorphism (a degenerate case whose square root happens to land on the correct branch).

Recommendation. Add a remark in Algorithm 3.15 or in the description of the $(2,2)$-isogeny chain that the difference $\theta(P)-\theta(Q)$ must be computed by applying θ to $P-Q$ directly (via the biladder with the same basis triple), *not* by recomputing DifferencePoint on the outputs. The same constraint applies to every other site that constructs a basis triple for the Okeya-Sakurai lift.

K GetIndexSplitting must handle malformed input

Affects Algorithm 8.42 (GetIndexSplitting), and transitively verification (Algorithm 4.9). Medium

Algorithm 8.42 searches the null point $U_{i,j}(0)$ for the unique index (i, j) where it vanishes. The spec states this index "exists and is unique" (Proposition 8.5.3), which is true for honestly generated signatures.

What goes wrong. For *malformed* signatures (corrupted `E_aux`, `M_chl`, or `chl`), the $(2,2)$-isogeny chain may produce a null point with zero or multiple vanishing entries. An implementation that asserts uniqueness (e.g., via `debug_assert!`) will panic on such inputs, providing a denial-of-service vector: any untrusted signature can crash the verifier in debug builds. In release builds, the

assertion compiles out and the function returns a wrong index, leading to silent misbehavior rather than a clean rejection.

We discovered this via negative test vectors (§J): single-bit flips in `E_aux`, `M_chl`, and `chl` all triggered the assertion.

Recommendation. Note in Algorithm 8.42 that implementations must handle the case where no (or multiple) vanishing indices exist, returning an error rather than assuming uniqueness. A verification implementation should treat a missing or ambiguous splitting index as signature rejection, not an assertion failure.

Signing key encoding requires caching the ideal generator

Severity. Low (implementation note).

Affected algorithm. Key encoding (§4.6).

Current spec text. The signing key wire format encodes the generator α of the secret ideal $I_{\text{sk}} = O_0\langle\alpha, N\rangle$ as four signed 32-byte coordinates. The ideal is defined by α and the norm N; α is part of the key.

What goes wrong. An implementation that converts the ideal to HNF for internal use (as we do for lattice operations) loses the original generator. Recovering α from HNF requires a brute-force search over small linear combinations of basis vectors. The spec does not mention this recovery problem.

Recommendation. Note in §4.6 that implementations should cache α at parse/keygen time rather than attempting to recover it from the lattice representation.

L IdealToIsogeny line 6: matrix formula is under-specified

Affects Algorithm 3.13 (IdealToIsogeny), line 6. HIGH

Algorithm 3.13 line 6 writes

$$[P, Q]^T \leftarrow \tfrac{1}{\text{nrd}(I)\,\text{nrd}(J_t)}\, M_{\beta_1^{-1}}\, M_{\beta_2}\, [\varphi_v(P_t), \varphi_v(Q_t)]^T.$$

The β_1^{-1} factor is a quaternion inverse, not a matrix inverse. The identity derived in the surrounding text,

$$\beta_2 \circ \varphi_{J_t} \circ \widehat{\varphi_{J_s}} \circ \widehat{\beta_1} = [\text{nrd}(I)\,\text{nrd}(J_s)\,\text{nrd}(J_t)]\widehat{\varphi_{I_2}} \circ \varphi_{I_1},$$

uses the conjugate $\widehat{\beta_1}$, and $\beta_1^{-1} = \widehat{\beta_1}/\text{nrd}(\beta_1)$ only as quaternions. An implementor writing $M_{\beta_1^{-1}}$ as a *matrix inverse* gets a different matrix from the intended $M_{\widehat{\beta_1}}/\text{nrd}(\beta_1) = \text{adj}(M_{\beta_1})/\text{nrd}(\beta_1) \pmod{2^f}$ in general.

What the C reference does. `dim2id2iso.c:881–1014` builds the quaternion $\theta = \beta_2 \cdot \widehat{\beta_1}$ directly and scales its coordinates by $\text{invmod}(d_1 \cdot \text{nrd}(J_t), 2^f)$ before decomposing on O_0. This is $\beta_2 \cdot \widehat{\beta_1}$, whereas the spec's $\beta_1^{-1}\,\beta_2 = \widehat{\beta_1}\,\beta_2/\text{nrd}(\beta_1)$ has the factors in the opposite order; the two quaternion products agree only after the subsequent $\text{nrd}(\beta_1)$ scaling, and quaternion multiplication is not commutative in general.

Impact. An implementor has two independent traps: (a) interpreting $M_{\beta_1^{-1}}$ as the matrix inverse rather than the action matrix of β_1^{-1} viewed as a quaternion; (b) writing the quaternion product in the opposite order from the derivation.

Recommendation. Write line 6 with the conjugate factored out and the scaling collected:

$$[P, Q]^T \leftarrow \tfrac{1}{\mathrm{nrd}(I)\,\mathrm{nrd}(J_t)\,\mathrm{nrd}(\beta_1)} M_{\widehat{\beta_1}}\, M_{\beta_2}\, [\varphi_v(P_t), \varphi_v(Q_t)]^T,$$

and state which quaternion-product order the matrix composition uses (equivalently, specify whether $M_{\alpha\beta} = M_\alpha M_\beta$ or $M_\beta M_\alpha$).

M IdealToIsogeny: β lives in the reduced ideal, not the input I

Affects Algorithm 3.13, Algorithm 3.16 (SuitableIdeals), and anywhere the scaling $1/\mathrm{nrd}(I)$ *appears downstream of short-vector enumeration.* High

Algorithm 3.16 (SuitableIdeals) returns β_i obtained from the short-vector enumeration inside $J_i\, I$, and $d_i = \mathrm{nrd}(\beta_i)/\mathrm{nrd}(J_i\, I)$. Efficient implementations (and the C reference, `dim2id2iso.c:534–565`) first replace I by the *smallest equivalent ideal* $I' = I \cdot \widehat{\delta}/\mathrm{nrd}(I)$, so that the short-vector search produces usable degrees. The returned β and d are relative to I', not to I, so the downstream invariant is

$$\mathrm{nrd}(\beta) = d \cdot \mathrm{nrd}(I') \neq d \cdot \mathrm{nrd}(I).$$

What the C reference does. `dim2id2iso.c:670–684` "sends β back" to the original ideal after the pair is selected (multiplying by δ), so that by the time Algorithm 3.13 line 6 applies the scaling $1/\mathrm{nrd}(I)$, the identity $\mathrm{nrd}(\beta_1) = d_1 \cdot \mathrm{nrd}(I)$ holds in terms of the caller-supplied I.

What goes wrong. An implementor who keeps β in the reduced ideal I' (a natural data representation) must also use $\mathrm{nrd}(I')$ in every downstream scaling formula. Using the spec's $\mathrm{nrd}(I)$ literal gives $\det(M_{\beta_1}) \neq d_1 \cdot \mathrm{nrd}(I) \pmod{2^f}$, silently producing a non-isotropic outer kernel and `zeros=0` in the final splitting step.

Recommendation. State in Algorithm 3.16 that the returned β must be expressed in the input ideal's lattice (so $\mathrm{nrd}(\beta) = d \cdot \mathrm{nrd}(I)$), or name the lattice that β lives in and refer to its norm throughout Algorithm 3.13.

N SuitableIdeals line 14: $v_2(u)$ vs. $v_2(\gcd(u, v))$

Affects Algorithm 3.16 (SuitableIdeals), line 14. Medium

Algorithm 3.16 line 14 writes "$e \leftarrow v_2(u)$" and line 15 divides both u and v by 2^e. Using $v_2(u)$ is only equivalent to $v_2(\gcd(u, v))$ when $v_2(u) \leq v_2(v)$:

- If $v_2(u) > v_2(v)$, dividing v by $2^{v_2(u)}$ loses low bits of v that were carrying real value.
- If $v_2(u) < v_2(v)$, dividing by $2^{v_2(u)}$ leaves v still even, so the next algorithm's precondition $\gcd(u \cdot d_1, v \cdot d_2) = 1$ fails.

What the C reference does. `dim2id2iso.c:833` computes `ibz_two_adic(&gcd(u, v))`, i.e. $v_2(\gcd(u, v))$.

Recommendation. Write line 14 as "$e \leftarrow v_2(\gcd(u, v))$".

O Algorithm 8.47 implementation variants are not specified

Affects Algorithm 8.47 (IsogenyTwoChainWithTorsion) and its callers, primarily Algorithm 3.13 line 9. Observed 99%-wrong-codomain failure rate in keygen before the implementation accounted for the kernel-torsion convention. HIGH

The spec describes a single uniform chain of e 8-torsion isogenies, with the phrasing "a chain of e $(2,2)$-isogenies of 2-power degree." It does not make explicit either

1. what kernel torsion the chain consumes, nor
2. that Algorithm 3.13 and Algorithm 3.15 call the chain with different conventions.

The C reference implementation in `theta_isogenies.c` splits Algorithm 8.47 into two variants keyed on a boolean `extra_torsion`:

- `extra_torsion = true`: an e-length chain of 8-torsion isogenies. Kernel must have order 2^{e+2}. The last two chain steps use the `hadamard_bool` convention from Algorithm 8.41 to fold the kernel's 4-torsion and 2-torsion residues into the penultimate and ultimate $(2,2)$-isogenies.
- `extra_torsion = false`: an $(e-2)$-length chain of 8-torsion isogenies, followed by a dedicated 4-isogeny step (`theta_isogeny_compute_4`) and a dedicated 2-isogeny step (`theta_isogeny_compute_2`). Consumes a kernel of order 2^e. The final two steps compute square roots rather than relying on extra torsion.

Algorithm 3.13 line 9 (the outer chain) calls the implementation with `extra_torsion = false` (`dim2id2iso.c:1128`); the inner Algorithm 3.15 calls it with `extra_torsion = true` (`dim2id2iso.c:181`). The two variants produce the same abstract $(2^e, 2^e)$-isogeny but require different kernel torsions.

Concrete impact (Selkie experience). We implemented the 8-torsion chain corresponding to `extra_torsion = true` and used it for every call. Algorithm 3.13 line 9 passes a kernel whose natural order is 2^e (from doubling the 2^f-torsion image basis by $f-e$), so our chain received a kernel that was two torsion bits short of what the `extra_torsion = true` path expects. The chain ran to completion in every instance, the splitting code (Algorithm 8.41) silently selected one of the ten $U_{i,j}(0)$ indices, and keygen returned a curve that is not the public key E_{pk}. 100% of outer chains produced terminal splits with either 0 or 10 vanishing $U_{i,j}(0)$ coordinates rather than the unique 1 that identifies a genuine elliptic product. Downstream code paths did not detect the failure: Algorithm 4.3, Algorithm 4.6, and Algorithm 4.9 all accept whichever curve the chain produces.

We resolved this by over-padding the kernel: instead of doubling $f-e$ times, we double $f-e-2$ times, so the kernel enters the chain with order 2^{e+2} and the `extra_torsion = true` path works unchanged. This requires $e \le f-2$. The side condition reduces to $v_2(\gcd(u,v)) \ge 2$ in FIND_UV_FROM_LISTS, which we filter for in Algorithm 3.16.

Why this is hard to catch in testing.

- The chain does not fault when given a short kernel; it returns a malformed abelian surface.
- The splitting routine Algorithm 8.42 (SplittingIsomorphism) picks a codomain side by locating a zero among the ten $U_{i,j}(0)$ coordinates and does not check that exactly one vanishes. When all ten vanish (the fully degenerate case), any of the ten branches is "valid"; the spec doesn't say which to pick. When none vanish, the function should error but the text only says "find the unique index such that $U_{i,j}(0) = 0$."

- Downstream consumers of the split codomain (basis computation in Algorithm 2.3, Tate pairing in Algorithm 2.5, kernel-to-ideal conversion in Algorithm 3.17) accept non-product curves without visible failure. They compute *something*, but it's not the object the protocol calls for.
- Only byte-for-byte comparison with KAT vectors reveals the divergence; unit tests on synthetic kernels with small e do not exhibit the bug because $f - e \gg 2$ leaves enough spare torsion implicitly.

Recommendation. We suggest three spec-level changes, in decreasing order of priority:

1. Algorithm 8.47 should state the kernel-torsion requirement explicitly. "A chain of e $(2,2)$-isogenies consumes a kernel of order 2^{e+2}" (current implicit convention) or "of order 2^e" (shorter variant) unambiguously pins down the caller's contract.
2. Algorithm 8.41 (SplittingIsomorphism) or Algorithm 8.42 should add a precondition: *exactly one* $U_{i,j}(0)$ must vanish at the chain's terminal theta null. An implementer who checks this invariant gets a clear failure signal instead of silent corruption.
3. Algorithm 3.13 line 9 should state explicitly which chain variant it uses, so that an implementer who supplies only one variant knows the coupling fails and either adapts (pad kernel up, pad chain down) or implements the missing variant.

Absent these changes, a faithful implementation that reads the spec straight through will silently produce invalid signing keys that fail only when compared against known-answer test vectors.

P IdealToIsogeny output basis must be aligned with the isogeny, not canonical on the codomain

Affects Algorithm 3.13 (IdealToIsogeny) and Algorithm 4.5 (SplitAuxiliaryIsogeny). High

Algorithm 3.13 states that its output is "the codomain E_I of φ_I, $\varphi_I(P_0)$, and $\varphi_I(Q_0)$." Read literally, this is the image of the canonical $\mathcal{O}_0$ basis (P_0, Q_0) of $E_0[2^f]$ through the full ideal-to-isogeny φ_I.

The implicit requirement. Algorithm 4.5 (SplitAuxiliaryIsogeny) takes as input curves E_1, E_2 and bases (P_1, Q_1), (P_2, Q_2) together with "positive integers q, e', r such that there exist four isogenies φ_q, $\varphi_{2^{e'}-q}$, $\psi_{2^{e'}-q}$, ψ_q verifying … $(P_2, Q_2) = (\varphi(P_1), \varphi(Q_1))$." The condition $(P_2, Q_2) = \varphi(P_1, Q_1)$ forces (P_2, Q_2) to be the image of (P_1, Q_1) through the specific odd-degree isogeny φ of degree $q\,(2^{e'} - q)$. An independently-chosen torsion basis on E_2 does not satisfy this and produces a kernel that fails the splitting condition inside Algorithm 8.47.

Why this is implicit. The spec states the output of Algorithm 3.13 as "$\varphi_I(P_0)$, $\varphi_I(Q_0)$." The composition $\varphi_I = \varphi_{2^e} \circ \cdots \circ \varphi_d$ has both odd-degree and 2-power factors. The image of P_0 through the full composition retains 2^f-torsion only because every odd-degree factor preserves the 2^f-torsion subgroup; the 2-power factors are evaluated through the dimension-2 $(2,2)$-chain on the abelian-surface side, where the input torsion is determined by what the implementer pushes through the chain. The C reference's Clapotis algorithm (`dim2id2iso_ideal_to_isogeny_clapotis` in `dim2id2iso.c:778–1228`) navigates this by pushing the canonical basis of the *intermediate* extremal-order curve E_t through the $(2,2)$-chain after the FixedDegreeIsogeny step has already produced an aligned basis on the chain's input curve, E_u. The final output basis on E_I is then the image of E_t's basis (not E_0's) through the chain, which is consistent with the alignment required by Algorithm 4.5. None of this is in the spec.

Impact. A naive implementation following Algorithm 3.13 returns either (a) the literal $\varphi_I(P_0), \varphi_I(Q_0)$ via push-through of E_0's basis through the entire chain — the resulting points have torsion $2^{f-2\,\mathsf{chain\_e}}$, which is trivial for realistic parameters where chain_e approaches f — or (b) a freshly-sampled canonical 2^f-torsion basis on E_I unrelated to φ_I. Either choice causes Algorithm 4.5 to silently fail at the $(2,2)$-chain splitting check.

What the C reference does. The C reference's Clapotis (`dim2id2iso_ideal_to_isogeny_clapotis`) effectively treats the ideal-to-isogeny output basis as "aligned with φ_I in a way that downstream Algorithm 4.5 can consume." Concretely it takes the canonical basis of the matching extremal-order curve E_t, pushes it through FixedDegreeIsogeny (an odd-degree isogeny φ_v that preserves 2^f-torsion), applies the endomorphism $\theta = \beta_2\,\widehat{\beta_1}$ to get a kernel structure on $E_u \times E_v$, runs the $(2,2)$-chain with the basis-preserving push-through, and reports the surviving basis on the chain's first codomain factor.

Recommendation. The spec should:

1. State explicitly that the output of Algorithm 3.13 is a pair $(R,S) \in E_I[2^f]^2$ such that $(R,S) = \nu(P_0,Q_0)$ for an isogeny ν *whose 2-adic valuation* the caller can recover from $(\beta_1,\beta_2,u,v,d_1,d_2)$, and pin down the convention so Algorithm 4.5's precondition $(P_2,Q_2) = \varphi(P_1,Q_1)$ is realized.
2. Reference the C reference's basis-alignment construction (or describe it inline) so an implementor doesn't read the *output* of Algorithm 3.13 as "the literal image of E_0's basis through the full chain" — which has the wrong torsion — nor as "a fresh canonical basis on E_I" — which has the right torsion but no alignment with φ.

We follow the C reference's basis-alignment construction in our implementation; the resulting basis is what Algorithm 4.5 consumes without further adjustment.

Q IdealToIsogeny must propagate $P-Q$ alongside (P,Q)

Affects Algorithm 3.13 (IdealToIsogeny) output and Algorithm 4.5 (SplitAuxiliaryIsogeny) input. Critical

Algorithm 3.13 writes its output as "$E_I, \varphi_I(P_0), \varphi_I(Q_0)$"—a curve and two points, no third coordinate. But Algorithm 4.5 forms a $(2,2)$-kernel from those two points (line 3, "$T \leftarrow [2^{f-e'}](\varphi_{\mathrm{com}}(P_0), [q_{\mathrm{rsp}}^{-1}\,2^{f-e'}]\varphi_{\mathrm{com,rsp,au}}$ and any $(2,2)$-chain that consumes such a kernel must call lift_basis (Okeya–Sakurai) to recover the Jacobian y-coordinate. Okeya–Sakurai requires a third basis component $P-Q$ *whose projective representative is consistent with the chain's prior evaluations of* P *and* Q — see §A and §I for the already-noted instances of the same pitfall.

If Algorithm 3.13 returns only $(\varphi_I(P_0), \varphi_I(Q_0))$ and the next stage recovers $\varphi_I(P_0 - Q_0)$ via DifferencePoint, the fresh square root branches independently of the chain's internal lift. The downstream $(2,2)$-chain (here the response-phase chain inside Algorithm 4.5) then reaches its terminal theta null with $\#\{i,j : U_{i,j}(0) = 0\} = 0$ instead of the unique 1 that identifies a product splitting (§??), and the chain fails.

What the C reference does. A C-reference `theta_couple_curve_with_basis_t` always carries $B.\mathrm{PmQ}$ alongside $B.P$ and $B.Q$, and every isogeny evaluation pushes all three through together. `dim2id2iso_ideal_to_isogeny_clapotis` (`dim2id2iso.c:778–1228`) does so for the Algorithm 3.13 chain, and the `copy_bases_to_kernel` helper inside `compute_dim2_isogeny_challenge` (`sign.c:240–256`) carries PmQ into the response-phase kernel. No conversion to/from DifferencePoint ever occurs after the canonical basis is lifted.

Impact. A literal reading of Algorithm 3.13's output triple $(E_I, \varphi_I(P_0), \varphi_I(Q_0))$ leads an implementor to recover $P-Q$ via DifferencePoint at the next call site, which silently corrupts Algorithm 4.5 on every input, with no error message: the chain returns "no splitting found" which is indistinguishable from a probabilistic chain failure that should trigger a retry. The implementor concludes the input ideals are pathological rather than that the basis is malformed.

Recommendation. Make explicit in Algorithm 3.13 that the output is a triple $(R, S, R-S) \in E_I[2^f]^3$, not a pair, and that the third component must be the propagated image $\varphi_I(P_0 - Q_0)$ of the canonical basis difference — never recovered via DifferencePoint from R and S. Add a corresponding note to Algorithm 4.5 that its input bases on E_1 and E_2 each include a propagated PmQ, and that the kernel construction (lines 1–4) must scale PmQ alongside P and Q rather than recomputing. Equivalently, update the TorsionBasisFromHint and IdealToIsogeny signatures to return the standard $(P, Q, P-Q)$ triple that §2 already implicitly assumes throughout.

R SplittingIsomorphism omits a side-channel randomization that the published KAT vectors require

Affects Algorithm 8.42 (SplittingIsomorphism) and the chain output of any algorithm that consumes it (Algorithm 3.13, Algorithm 4.5). High

The output of an extra-torsion-free $(2, 2)$-isogeny chain is a level-2 theta null point on a product abelian surface $E_1 \times E_2$. Multiple projective representatives exist for the same abstract surface, related by the symplectic modular-group action. Algorithm 8.42 as written picks one such representative deterministically from the chain's terminal theta null. But the choice is a function of the secret kernel, so the emitted projective representative leaks information about the kernel beyond what the curve isomorphism class already exposes.

What the C reference does. The C reference closes this leak in `splitting_compute` (`theta_isogenies.c:100` by pre-multiplying the deterministic splitting matrix M by a uniformly random M_k drawn from a precomputed list of six 4×4 matrices over $\mathbb{F}_{p^2}$. The six matrices are tensor products $M_k = m_k \otimes m_k$ of 2×2 coset representatives of the symplectic-modular-group quotient $\Gamma/\Gamma^0(4)$ that, as a group, suffice to mix the representative within its equivalence class when working in the Montgomery model. The matrices are built by `precompute_hd_splitting.sage` and shipped as `NORMALIZATION_TRANSFORMS` in `precomp/.../hd_splitting_transforms.c`.

The index $k \in \{0, \ldots, 5\}$ is sampled by reading four bytes from the deterministic byte-replayable RNG, parsing them as a little-endian `u32`, rejection-sampling against threshold $4{,}294{,}967{,}292 = 6 \cdot \lfloor 2^{32}/6 \rfloor$ for an unbiased mod 6, and indexing the precomputed list (`theta_isogenies.c:980`, `sample_random_index`). The randomization is gated on `#if defined(ENABLE_SIGN)` and on a caller-supplied `randomize` flag: it is on for the keygen and signing outer chains (`theta_chain_compute_and_eval_` and off for verification (`theta_chain_compute_and_eval_verify`) and for the deterministic FixedDegreeIsogeny chains (`theta_chain_compute_and_eval`).

Two consequences.

1. It is a real side-channel hardening. Without it, the public output of keygen and signing is a deterministic function of the secret kernel structure, not just the abstract codomain. An attacker observing the projective representative of a public key, signature, or commitment curve could potentially recover information about the underlying kernel beyond what the curve isomorphism class already reveals. Whether the leakage is exploitable in practice for SQIsign is, to our knowledge, an open question, but the cost of the countermeasure is six precomputed matrices, four bytes of RNG, and one matrix multiplication.

2. The published KAT vectors are produced *with* this randomization applied. An implementation that follows the spec literally and skips the randomization will produce a public key that is projectively equivalent to the KAT vector's, but byte-different. Without mirroring the exact same RNG byte consumption pattern (4 bytes, little-endian, threshold 4,294,967,292, mod 6) and matrix selection logic, byte-equality with the published KATs is impossible. We spent significant effort tracing chain-internal byte mismatches that vanished only after replicating the splitter randomization.

Recommendation. Add an explicit SplittingNormalization step to Algorithm 8.42 (or as a post-processing step on its output) that specifies:

- The list of six normalization matrices $M_0, \ldots, M_5$ (or a generating procedure — the Sage script that produces them is short).
- The exact RNG byte consumption pattern (four bytes, little-endian `u32`, rejection threshold 4,294,967,292, mod 6).
- A precise statement of which chain calls (keygen and signing outer chains, but not FDI sub-chains and not verification) randomize, and which use the deterministic form.
- The threat model: that the deterministic representative leaks kernel structure, and that the randomization is required to prevent that leak.

Without these specifics, an implementation that follows the spec correctly will still fail the KAT comparison and the implementor will have no way to discover the discrepancy without reading the C reference's `splitting_compute`.

S SuitableIdeals multi-order optimization: the cross-order β transport step

Affects Algorithm 3.16 (SuitableIdeals) when extended with the multi-order optimization, and any caller that expects $\beta_i \in I$ (the input ideal). Causes Algorithm 3.13 (IdealToIsogeny) to silently fail when the special-order pair is unavailable. High

Algorithm 3.16 as written enumerates short vectors only in the input ideal I (or its LLL-reduced equivalent I'). When this single-lattice search exhausts without finding a viable (β_s, β_t) pair, the C reference falls back to the *multi-order* optimization (`dim2id2iso.c:534–666`): it enumerates short vectors in six additional pushforward lattices

$$L_t = \overline{I'} \cdot J_t, \qquad t = 1, \ldots, 6,$$

where J_t is the precomputed connecting ideal from $\mathcal{O}_0$ to a maximal order $\mathcal{O}_t$ (the right order of J_t is conjugate to $\mathcal{O}_t$). This more-than-doubles the candidate pool and rescues KATs whose hypercube enumeration would otherwise exhaust.

A short vector $\beta_t \in L_t$ is an element of $\overline{I'} \cdot J_t \subseteq B$, *not* of I. Every downstream caller of SuitableIdeals (Algorithm 3.13, Algorithm 3.17, Algorithm 4.6) expects the returned β_i to live in the original ideal lattice I so they can compute $\theta = \beta_2 \cdot \overline{\beta_1}/\mathrm{nrd}(I)$ and treat it as an endomorphism of the protocol's reference curve. The multi-order optimization therefore needs an explicit transport step from L_t back to I before returning β_i.

What the C reference does. `dim2id2iso.c:651–672`, after `find_uv_from_lists` selects an (i_1, i_2) pair, applies the following transformation when either index j_1 or j_2 is non-zero (i.e., the chosen β came from an alternate-order lattice):

1. Adjust `delta.denom`, where `delta` is the conjugated LLL-first vector $\overline{\delta}$ of I':

   ```
   ibz_div(&delta.denom, &remain, &delta.denom, &lideal->norm);
   ibz_mul(&delta.denom, &delta.denom, &conj_ideal.norm);
   ```

   so that `delta` ends up at denom $d_{I'} \cdot \mathrm{nrd}(\overline{I'})$.

2. For each $j_i \neq 0$:

   ```
   quat_alg_mul(beta_i, &delta, beta_i, Bpoo);
   quat_alg_normalize(beta_i);
   quat_alg_conj(beta_i, beta_i);
   ```

Algebraically: $\beta_i \leftarrow \overline{\overline{\delta} \cdot \beta_i}$. The post-condition asserted at `dim2id2iso.c:690` is $\beta_i \in I$ (i.e., `quat_lattice_contains(&I, beta_i)`).

This step is not in Algorithm 3.16's pseudocode at all. An implementor reading the spec will write SuitableIdeals without this transport, observe that 99% of NIST-I KATs pass (because SuitableIdeals usually finds a special-order pair on the first try), and only discover the bug on a KAT that forces the multi-order branch. In the published NIST-I lvl1 KAT file, that's count 29: every other count from 0 through 99 finds a special-order pair within the `FINDUV_box_size = 2` hypercube.

Concrete impact (Selkie experience). Without the transport, our β_2 from $L_1 = \overline{I'} \cdot J_1$ flowed into the downstream `action_matrix`(β_2, O_1) call, whose `decompose` step rejects β_2 as "not in the lattice" and aborts the entire Algorithm 3.13 flow with no error signal — key generation transparently retries from the next random commitment, eventually picks an $(s, t) = (0, 0)$ pair on some later iter, and produces a public key that fails the KAT byte comparison.

Recommendation. Algorithm 3.16 should be extended to describe the multi-order optimization explicitly, including the post-search transport. Concretely:

1. **Search step.** Define $L_0 = I'$ and $L_t = \overline{I'} \cdot J_t$ for $t = 1, \ldots, 6$, where J_t is the connecting ideal from O_0 to O_t. Enumerate short vectors in each L_t separately; sort each list by reduced norm.

2. **Pair selection.** Run the line-walk in `find_uv_from_lists` over the cross product of sorted lists. First valid $(\beta_s \in L_s, \beta_t \in L_t)$ wins.

3. **Transport step.** If $s \neq 0$, replace $\beta_s \leftarrow \overline{\overline{\delta} \cdot \beta_s}$; if $t \neq 0$, replace $\beta_t \leftarrow \overline{\overline{\delta} \cdot \beta_t}$. Here $\overline{\delta}$ is the conjugate of the LLL-first vector of I', with denominator $d_{I'} \cdot N(\overline{I'})$ after the modular adjustment described above.

The transport's algebraic role is to map an $\overline{I'} \cdot J_t$-element back to an I-element via the $\overline{I'} \cdot \overline{\delta} = N(\overline{I'}) \cdot \overline{I}$ identity (and then conjugate to flip back to a left-O_0 element).

The spec's silence on this transport is part of a broader pattern: the multi-order optimization is invoked extensively by the C reference but is not modeled in Algorithm 3.16 at all. Implementations that follow the spec literally end up with a single-lattice SuitableIdeals that fails on KATs where the multi-order search is needed.

T Connecting-ideal norm encoding: reduced norm vs. first basis entry

Affects the precomputation of the seven extremal-order connecting ideals J_t used by the multi-order SuitableIdeals *optimization (Algorithm 3.16) and by Algorithm 3.13 (*IdealToIsogeny*) line 6's outer-chain scaling.* High

The seven extremal maximal orders O_t come with precomputed connecting ideals $J_t \subseteq O_0$ whose right orders are conjugate to O_t. Each J_t is encoded as a basis matrix B_t, a denominator d_t, and a labelled norm N_t. The rational basis of J_t is given by the columns of B_t/d_t, and the labelled norm is supposed to be the *reduced norm* $N(J_t) = \sqrt{[O_0 : J_t]}$.

What goes wrong. For the HNF shape used by the NIST-I precomputed ideals ($B_t = \mathrm{diag}(2N_t, 2N_t, 1, 1)$ at $d_t = 2$), the reduced norm is N_t but the first-coordinate entry of the integer basis is $2N_t = d_t \cdot N_t$. An implementer who extracts the labelled norm by reading the leading entry of the basis ends up encoding $d_t \cdot N_t$ instead of N_t, i.e. the labelled norm is exactly d_t times too large for every cross-order $t > 0$.

Downstream impact.

- SuitableIdeals' integrality check ($\mathrm{nrd}(\beta)/(N_t \cdot d_t^2)$) silently rejects half the hypercube candidates and silently halves the degrees of the rest. The resulting sorted batch differs from the C reference's, and FIND_UV_FROM_LISTS picks a different (β_s, β_t).
- Algorithm 3.13 line 6's outer-chain scaling $1/(\mathrm{nrd}(I) \cdot d_1 \cdot N(J_t))$ for the cross-order case is multiplied by d_t, embedding the output basis in a sublattice of half the intended index.

This is latent on most KATs because the special-order path ($t = 0$, which doesn't read the labelled norm at all) usually succeeds. KATs that force the cross-order fallback expose both divergences in the same iteration.

Recommendation. The spec should define the connecting-ideal precomputation format explicitly: it is a triple (B_t, d_t, N_t) where

- B_t/d_t are the rational basis columns,
- N_t is the reduced norm $N(J_t)$ (*not* the leading basis entry, which equals $d_t \cdot N_t$ for the typical HNF shape).

The natural pitfall is to extract N_t as the first coordinate of the integer HNF row space, which yields a d_t-times-too-large value. Implementations should derive N_t via $\sqrt{|\det(B_t)|/d_t^4}$ or equivalently $\sqrt{[O_0 : J_t]}$, matching what `quat_lideal_norm` computes (`quaternion/ref/generic/ideal.c:7`).

U IdealToIsogeny action-matrix order index after cross-order β transport

*Affects Algorithm 3.13 (*IdealToIsogeny*) when its caller-provided (β_1, β_2) came from the multi-order optimization in* SuitableIdeals*.* Medium

After the transport described in §R, both β_1 and β_2 live in $I \subseteq O_0$, regardless of which alternate-order lattice they originated from. An implementor reading Algorithm 3.13 naturally writes ActionByTranslation (line 7) to decompose β_i on the basis of O_s (resp. O_t), since the algorithm associates β_1 with the curve E_s and β_2 with E_t. This decomposition fails: β_i is in $I \subseteq O_0$, not in O_s (or O_t) when $s \neq 0$ (or $t \neq 0$).

What the C reference does. `endomorphism_application_even_basis` (`id2iso.c:140`) takes an explicit `index_alternate_curve` argument selecting which order's basis to decompose on. In every call from IdealToIsogeny, regardless of which alternate order β_i originated from, the C reference passes `0` (see `dim2id2iso.c:1054` and `:1156`):

```
endomorphism_application_even_basis(
    &bas2, 0, &Fv_codomain.E1, &theta, TORSION_EVEN_POWER);
```

i.e., always decompose on $\mathcal{O}_0$. This is consistent with the post-transport invariant $\beta_i \in I \subseteq \mathcal{O}_0$.

The two conventions (decompose on $\mathcal{O}_s$ vs. decompose on $\mathcal{O}_0$) are mathematically equivalent for $\beta \in \mathcal{O}_0 \cap \mathcal{O}_s$. But for cross-order β, only the $\mathcal{O}_0$ decomposition succeeds; an implementer who reads "M_β is the action of β on E_s" as "decompose β on $\mathcal{O}_s$" has the right idea for the $s = 0$ case and the wrong idea for $s \neq 0$.

Recommendation. Algorithm 3.13 should specify that the action-matrix decomposition is taken on $\mathcal{O}_0$ regardless of the source order. Equivalently: the ActionByTranslation subroutine should take both the curve and the order as arguments, and Algorithm 3.13 should always pass $(\mathcal{O}_0, E_*)$ for some target curve E_*. A one-line addition of the form

> *In line 7, the action-matrix decomposition $M_{\beta_i} = \sum_k c_k M_{b_k}$ is taken on the basis $(b_0, \ldots, b_3)$ of $\mathcal{O}_0$, where $\beta_i = \sum_k c_k b_k$ (which is well-defined because the multi-order SuitableIdeals returns $\beta_i \in I \subseteq \mathcal{O}_0$).*

suffices.

Why this is hard to catch. The two-decomposition factoring $M_{\beta_2} \cdot \mathrm{adj}(M_{\beta_1})$ that some implementations (including ours) use as an alternative to computing $\theta = \beta_2 \cdot \overline{\beta_1}/\mathrm{nrd}(I)$ as a quaternion does *not* work for cross-order pairs: it requires both M_{β_i} to be well-defined relative to a common torsion basis, which the $\mathcal{O}_s, \mathcal{O}_t$ decompositions are not. An implementer who chooses this factoring will pass every single-order KAT and hit the cross-order path as a hard Decompose-returns-None error rather than a quiet wrong-result; this is a comparatively kind failure mode, but only because both bugs cancel. An implementer who instead follows Algorithm 3.13's natural reading and decomposes on $\mathcal{O}_s, \mathcal{O}_t$ directly hits the same error. Either way, the spec sends the implementer to the C reference to discover the always-decompose-on-$\mathcal{O}_0$ convention.